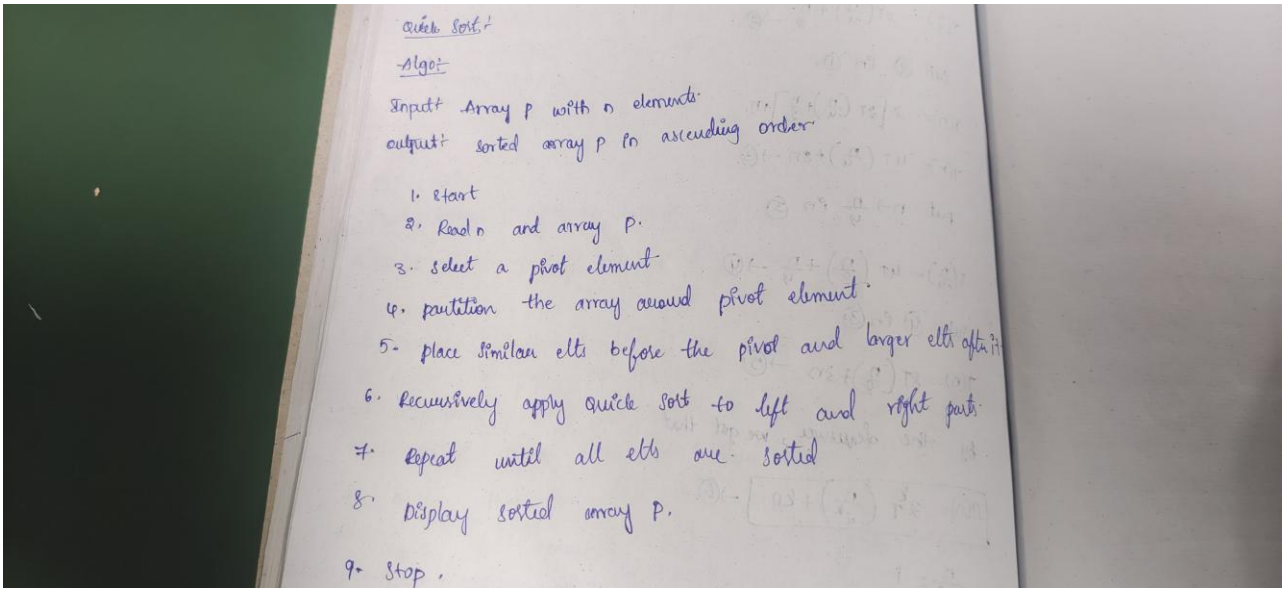
Aim: To Implementation and time analysis of Quick sort algorithm.

Algorithm for Quick sort:



Program of Quick sort:

```
#include <iostream>
using namespace std;

class QuickSortApp
{
public:
   int arr[100], n;

   void arrayInput()
   {
      cout << "Enter number of elements: ";
      cin >> n;

      cout << "Enter the elements: ";
      for (int i = 0; i < n; i++)
      {
```

```
        cin >> arr[i];
    }
}

void printArray()
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

int partition(int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;

            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;

    return i + 1;
}

void quickSort(int low, int high)
```

```
    {
        if (low < high)
        {
            int pivotIndex = partition(low, high);

            quickSort(low, pivotIndex - 1);
            quickSort(pivotIndex + 1, high);
        }
    }

    // Quick Sort Function
    void sort()
    {
        quickSort(0, n - 1);
    }
};

int main()
{
    QuickSortApp qs;

    qs.arrayInput();

    cout << "\nArray before sorting: ";
    qs.printArray();

    qs.sort();

    cout << "\nArray after Quick Sort: ";
    qs.printArray();

    return 0;
}
```

Output:

```
Output
Enter number of elements: 5
Enter the elements: 2
1
8
4
3

Array before sorting: 2 1 8 4 3

Array after Selection Sort: 1 2 3 4 8


=== Code Execution Successful ===
```

Time complexity: